



Geração aumentada por recuperação

A **geração aumentada por recuperação** (do inglês **retrieval-augmented generation**, sigla **RAG**) é uma técnica de inteligência artificial que aprimora modelos de linguagem de grande escala (LLMs) incorporando um mecanismo de recuperação de informações que permite ao modelo acessar e utilizar dados adicionais além do seu conjunto de treinamento original.^[1] Com o RAG, os LLMs primeiro consultam um conjunto especificado de documentos externos antes de responder a consultas dos usuários, complementando o conhecimento paramétrico armazenado nos pesos do modelo com uma memória não paramétrica atualizada e verificável.^[2]

O termo foi cunhado por Patrick Lewis e colegas da Meta AI (então Facebook AI Research), da University College London e da New York University, em um artigo seminal apresentado na conferência NeurIPS em 2020.^[3] O próprio Lewis reconheceu, em entrevista à NVIDIA, que o acrônimo RAG não era o mais elegante, comentando que "definitivamente teríamos pensado mais no nome se soubéssemos que nosso trabalho se tornaria tão difundido".^[3]

O RAG tornou-se uma das técnicas mais amplamente adotadas na engenharia de aplicações de IA generativa, sendo utilizado por empresas como AWS, IBM, Google, Microsoft, NVIDIA e Oracle, entre outras.^[3] Em essência, o RAG é uma forma de melhorar o desempenho dos LLMs combinando o processo de geração do modelo com uma busca em documentos ou bases de dados, ajudando os modelos a se manterem fiéis aos fatos.^[4]

Contexto e motivação

Os modelos de linguagem de grande escala são treinados sobre vastos corpora de texto, internalizando conhecimento em seus parâmetros, fenômeno referido como **memória paramétrica**. Esse tipo de memória apresenta duas limitações fundamentais: os dados de treinamento possuem uma data de corte (*knowledge cutoff*), tornando o modelo incapaz de responder com precisão sobre eventos posteriores ao treinamento, e o modelo não tem acesso a informações privadas ou proprietárias de uma organização específica que não façam parte do corpus de treinamento público.^[2]

Quando um LLM tenta responder a perguntas sobre domínios que não estão presentes em seus dados de treinamento, ou que exigem informações recentes, o modelo pode gerar respostas plausíveis, porém incorretas, fenômeno denominado **alucinação**.^[3] Um exemplo amplamente citado ocorreu quando o Google demonstrou o LLM Google Bard (posteriormente rebatizado como Gemini), que forneceu informações incorretas sobre o Telescópio Espacial James Webb, contribuindo para uma queda de US\$ 100 bilhões na capitalização de mercado da empresa.^[5]

O RAG foi proposto como solução para essas limitações. Em vez de retrainar o modelo com novos dados, o que é computacionalmente custoso, o RAG permite que o modelo consulte dinamicamente uma base de conhecimento externa durante a geração de respostas. Quando novas informações se tornam disponíveis,

basta atualizar a base de conhecimento externa, sem necessidade de retreinar o modelo.^[4] O RAG foi descrito pelos autores originais como uma "receita de ajuste fino de propósito geral", pois pode ser utilizado por praticamente qualquer LLM para conectar-se a quase qualquer recurso externo.^[3]

Arquitetura e funcionamento

Visão geral do pipeline

O pipeline RAG é composto por três etapas fundamentais: **recuperação** (*retrieval*), **augmentação** (*augmentation*) e **geração** (*generation*).^[2] A augmentação é tecnicamente a saída do recuperador e serve como entrada para o gerador, sendo o elo entre as duas outras etapas.

Quando um usuário submete uma consulta, ela é processada da seguinte forma:

1. A consulta é convertida em uma representação numérica chamada **embedding** ou **vetor**, produzida por um modelo de embedding especializado.
2. O sistema recuperador (*retriever*) usa esse vetor para buscar os documentos mais relevantes em uma base de conhecimento indexada, geralmente por meio de um **banco de dados vetorial**.
3. Os documentos recuperados são concatenados à consulta original, formando um contexto enriquecido que é fornecido ao LLM por meio de engenharia de prompt.
4. O LLM gera uma resposta baseada tanto no contexto recuperado quanto no seu conhecimento paramétrico, podendo citar as fontes utilizadas.^[6]

Indexação e chunking

Antes que a recuperação possa ocorrer, os documentos da base de conhecimento precisam ser indexados. Esse processo envolve dividir os documentos em fragmentos menores chamados **chunks**, convertê-los em embeddings e armazenar esses embeddings em um banco de dados vetorial. A escolha da estratégia de fragmentação (*chunking strategy*) é crítica para o desempenho do sistema: fragmentos muito pequenos perdem contexto, enquanto fragmentos muito grandes excedem a janela de contexto do LLM e introduzem ruído.^[7]

No artigo original de Lewis et al. (2020), cada artigo da Wikipédia foi dividido em fragmentos de 100 palavras, resultando em um total de 21 milhões de documentos indexados com o sistema FAISS (Facebook AI Similarity Search).^[1]

Recuperação: vetores densos e esparsos

A recuperação de documentos relevantes pode ser realizada por diferentes métodos, que dependem em parte do tipo de indexação utilizada.^{ref name="MIT Technology Review-2024">}«Why Google's AI Overviews gets things wrong» (<https://www.technologyreview.com/2024/05/31/1093019/why-are-google-s-ai-overviews-results-so-bad/>). *MIT Technology Review*. 31 maio 2024. Consultado em 7 março 2025</ref>^[8]

Os **vetores esparsos** codificam a identidade das palavras. Tipicamente têm comprimento de dicionário e contêm majoritariamente zeros. O método clássico baseado em vetores esparsos é o **BM25** (*Best Match 25*), uma função de ranqueamento de documentos amplamente utilizada em sistemas de busca tradicionais, que considera frequência de termos e comprimento dos documentos.^[7]

Os **vetores densos** codificam o significado semântico das palavras e frases. São mais compactos e contêm menos zeros. Modelos como o DPR (*Dense Passage Retriever*) e os *Sentence Transformers* convertem textos em vetores densos que capturam similaridade semântica mesmo quando as palavras exatas não coincidem. A busca por vizinhos mais próximos aproximados (ANN, *Approximate Nearest Neighbor*) melhora a eficiência em relação à busca exata de K vizinhos mais próximos (KNN).^[9]

A **recuperação híbrida** combina vetores esparsos e densos, aproveitando as vantagens de ambas as abordagens, geralmente por meio de algoritmos de fusão como o RRF (*Reciprocal Rank Fusion*).^[10]

Bancos de dados vetoriais

Um componente central da arquitetura RAG é o **banco de dados vetorial** (*vector database* ou *vector store*), que armazena embeddings de documentos e permite buscas eficientes por similaridade semântica. Os principais bancos de dados vetoriais utilizados em implementações RAG incluem:

- **FAISS** (Facebook AI Similarity Search): Biblioteca de código aberto da Meta AI para busca eficiente em grandes coleções de vetores densos, com suporte a índices HNSW (*Hierarchical Navigable Small World*) e IVFFlat.^[1]
- **Chroma** (ChromaDB): Banco de dados vetorial de código aberto, projetado para facilidade de uso em aplicações de IA.
- **Pinecone**: Serviço gerenciado de banco de dados vetorial, amplamente adotado em produção.
- **Weaviate**: Banco de dados vetorial de código aberto com suporte a busca híbrida nativa.
- **Qdrant**: Banco de dados vetorial de código aberto escrito em Rust, com foco em desempenho e filtragem avançada.
- **Elasticsearch**: Sistema de busca de código aberto que adicionou suporte nativo a vetores densos a partir da versão 8.0, permitindo busca híbrida em um único sistema.^[10]

Geração aumentada

Na fase de geração, o LLM recebe a consulta original combinada com os documentos recuperados, e produz uma resposta que integra tanto o conhecimento recuperado quanto o conhecimento paramétrico interno. Os LLMs com RAG são programados para priorizar as informações recuperadas, técnica por vezes chamada de **prompt stuffing** (ou preenchimento de prompt). A capacidade de citar as fontes utilizadas é uma das principais vantagens do RAG, pois permite que os usuários verifiquem as alegações feitas pelo modelo.^[3]

Variantes e evoluções

RAG ingênuo (Naive RAG)

O RAG ingênuo (*Naive RAG*) segue o processo básico de indexação, recuperação e geração descrito na seção anterior. Apresenta limitações como baixa precisão na recuperação (fragmentos recuperados desalinhados com a consulta), baixo recall (falha em recuperar todos os fragmentos relevantes), e possibilidade de fornecer ao LLM informações desatualizadas. Podem ocorrer também problemas de redundância e repetição quando múltiplas passagens recuperadas são utilizadas.^[7]

RAG avançado (Advanced RAG)

O RAG avançado (*Advanced RAG*) aborda as limitações do RAG ingênuo por meio de otimizações em três estágios:

- **Pré-recuperação:** Otimização da indexação de dados em cinco dimensões: granularidade dos dados, estruturas de índice, adição de metadados, otimização de alinhamento e recuperação mista.
- **Recuperação:** Ajuste fino do modelo de embedding para otimizar a relevância dos fragmentos recuperados, uso de embeddings dinâmicos que capturam melhor a compreensão contextual.
- **Pós-recuperação:** Reranking dos documentos recuperados, seleção de contexto relevante, compressão de informações redundantes e ajuste fino do gerador.^[7]

RAG modular (Modular RAG)

O RAG modular representa uma evolução mais flexível, na qual os componentes do pipeline são substituíveis e configuráveis de forma independente. Permite a incorporação de módulos especializados com capacidades como expansão de consultas em múltiplas dimensões, reescrita de consultas, recuperação iterativa e reflexão sobre as próprias respostas geradas.^{[11][12]}

Self-RAG

O **Self-RAG** (RAG com autorreflexão) é uma variante na qual o modelo aprende a recuperar, gerar e criticar suas próprias respostas por meio de reflexão. O sistema usa três prompts em sequência: gera uma resposta inicial com citações, lista afirmações sem citação, e refina a resposta usando apenas as passagens citadas. Estudos demonstraram que o Self-RAG reduz taxas de alucinação para 5,8% em cenários de suporte à decisão clínica.^[10]

GraphRAG

O **GraphRAG**, desenvolvido pela Microsoft Research, combina recuperação baseada em grafos de conhecimento com a arquitetura RAG tradicional. Ao estruturar o conhecimento como um grafo de entidades e relações, o GraphRAG melhora a capacidade de raciocínio sobre dados narrativos privados e relacionamentos complexos entre conceitos, sendo especialmente útil para análise de documentos corporativos.^[13]

RAPTOR

O **RAPTOR** (*Recursive Abstractive Processing for Tree-Organized Retrieval*) é uma extensão que processa textos de forma hierárquica, incorporando, agrupando e resumizando textos em múltiplos níveis de abstração recursivamente. Essa abordagem melhora o desempenho em tarefas complexas de raciocínio em múltiplos passos, que exigem integração de informações de diferentes partes de um documento longo.^[2]

RAFT

O **RAFT** (*Retrieval-Augmented Fine-Tuning*) combina as vantagens do RAG e do ajuste fino (*fine-tuning*), criando conjuntos de dados sintéticos para ajustar modelos a domínios específicos. O RAFT supera o RAG tradicional em domínios especializados como medicina, pois permite que o modelo "estude" o conhecimento do domínio com antecedência, resultando em melhor desempenho do que o RAG tradicional.^[2]

Vantagens

As principais vantagens do RAG em relação ao uso isolado de LLMs são:

Redução de alucinações: Ao fundamentar as respostas em documentos recuperados de fontes verificáveis, o RAG reduz significativamente a probabilidade de o modelo gerar respostas plausíveis, porém incorretas.^[1] Estudos comparativos demonstram reduções de alucinação superiores a 40% em aplicações médicas com variantes avançadas de RAG.^[14]

Acesso a informações atualizadas: Quando novas informações se tornam disponíveis, basta atualizar a base de conhecimento externa, sem necessidade de retreinar o modelo. Isso elimina o problema do corte de conhecimento (*knowledge cutoff*) dos LLMs.^[4]

Acesso a dados organizacionais privados: Empresas podem utilizar RAG para conectar LLMs a manuais técnicos, documentos internos, contratos e bases de dados proprietárias, sem exposição de dados sensíveis durante o treinamento do modelo.^[2]

Transparência e verificabilidade: O RAG fornece aos modelos fontes que podem ser citadas, como notas de rodapé em um artigo de pesquisa, permitindo que os usuários verifiquem as alegações feitas. Isso aumenta a confiança nas respostas geradas.^[3]

Custo menor do que o ajuste fino: O desenvolvimento de um banco de dados vetorial é muito menos intensivo em recursos computacionais do que o ajuste fino de um LLM, tornando o RAG acessível a organizações que não dispõem de infraestrutura para treinamento de modelos.^[2]

Controle de domínio: Sistemas baseados em RAG podem ser configurados para responder apenas a partir de fontes dentro de um domínio de interesse, excluindo domínios irrelevantes ou indesejados.^[2]

Limitações e desafios

Qualidade da recuperação

A eficácia do RAG depende diretamente da qualidade dos documentos recuperados. Quando nem todos os fragmentos relevantes são recuperados (baixo recall de contexto), o LLM pode produzir respostas incompletas. Quando os fragmentos recuperados não estão alinhados com a consulta do usuário (baixa precisão), podem ser introduzidas alucinações mesmo com informações factuais corretas, caso o modelo interprete equivocadamente o contexto.^[15]

Efeito "chunk com viseiras" (Blinkered Chunk Effect)

Klesel e Wittmann (2025) descrevem o problema denominado **efeito de chunk com viseiras** (*blinkered chunk effect*, BCE): quando um fragmento de texto é extraído de um documento maior sem o contexto circundante, o modelo pode ter dificuldade de compreendê-lo corretamente, da mesma forma que uma pessoa teria dificuldade de entender um parágrafo isolado de um romance sem ter lido o restante da obra.^[2]

Latência e custo computacional

A etapa adicional de recuperação introduz latência no pipeline de geração, o que pode ser problemático para aplicações em tempo real. Sistemas de recuperação esparsa oferecem tempos de resposta mais rápidos (da ordem de 120 ms), porém com menor precisão do que sistemas de recuperação densa ou híbrida.^[10]

Alucinações persistentes

O RAG reduz, mas não elimina completamente as alucinações. LLMs podem gerar desinformação mesmo ao consultar fontes factualmente corretas, caso interpretem equivocadamente o contexto dos documentos recuperados. Além disso, se as fontes da base de conhecimento contiverem informações contraditórias ou incorretas, o modelo pode propagar esses erros.^[11]

Qualidade dos dados da base de conhecimento

Manter uma base de conhecimento de alta qualidade exige esforço contínuo de curadoria. Casos em que as fontes de dados contêm informações contrafactuais ou incorretas representam um risco que deve ser gerenciado ativamente, com processos de validação e atualização regulares.^[2]

Aplicações

Suporte ao cliente e assistentes empresariais

O RAG é amplamente utilizado para criar assistentes de IA corporativos que respondem perguntas com base em documentos internos como manuais de política, bases de conhecimento de suporte técnico, catálogos de produtos e históricos de tickets. Isso permite que as empresas ofereçam assistentes altamente personalizados sem expor dados sensíveis durante o treinamento do modelo.^[3]

Saúde e medicina

Na área da saúde, o RAG é aplicado para conectar LLMs a bases de dados médicas, diretrizes clínicas, literatura científica e prontuários eletrônicos de pacientes. Sistemas RAG demonstraram capacidade de melhorar a triagem de pacientes para ensaios clínicos, reduzir custos e aumentar a eficiência em comparação com métodos manuais. Uma revisão sistemática de estudos publicados entre 2020 e 2025 identificou o RAG como a abordagem dominante para fundamentar LLMs em conhecimento médico.^[15]

Desenvolvimento de software e CVDS

No contexto do ciclo de desenvolvimento de software aumentado por IA, o RAG é empregado para conectar agentes de codificação a bases de conhecimento de projeto, documentação técnica, histórico de issues e decisões arquiteturais. Ferramentas como o Protocolo de Contexto de Modelo (MCP) podem atuar como camada de transporte padronizada entre agentes de IA e as fontes de dados RAG no ambiente de desenvolvimento, conectando recuperadores a repositórios de código, wikis de projeto e sistemas de gerenciamento de requisitos.^[16]

Sistemas de pergunta e resposta

O RAG foi originalmente proposto e avaliado em tarefas de pergunta e resposta em domínio aberto (*open-domain question answering*). Nos benchmarks originais de Lewis et al. (2020), o RAG demonstrou desempenho superior ao estado da arte em benchmarks como Natural Questions, WebQuestions e TriviaQA, além de melhorar resultados na verificação de fatos (FEVER) e na geração de perguntas estilo Jeopardy (MS-MARCO).^[1]

Análise jurídica e financeira

Analistas jurídicos e financeiros utilizam sistemas RAG para consultar grandes volumes de documentos como contratos, regulamentações, relatórios financeiros e decisões judiciais, obtendo respostas fundamentadas em fontes verificáveis que podem ser auditadas.^[3]

Acesso a dados em linguagem natural

O RAG habilita a consulta a bancos de dados estruturados em linguagem natural, permitindo que usuários não técnicos façam perguntas como "qual foi o faturamento por região no último trimestre?" sem precisar escrever SQL. Essa aplicação é frequentemente combinada com o Protocolo de Contexto de Modelo em

Ferramentas e frameworks

O ecossistema de ferramentas para construção de sistemas RAG cresceu significativamente entre 2022 e 2026. Os principais frameworks incluem:

- **LangChain**: Framework popular para construção de aplicações de LLM, com suporte nativo a pipelines RAG, integrações com dezenas de bancos de dados vetoriais e conectores para diversas fontes de dados.
- **LlamaIndex** (anteriormente GPT Index): Framework especializado em indexação e recuperação de dados para LLMs, com suporte a múltiplas estratégias de chunking, indexação hierárquica e integração com grafos de conhecimento.
- **Haystack** (deepset): Framework de código aberto para construção de pipelines de busca e RAG em nível de produção, com suporte a recuperação híbrida DPR + BM25 e reranqueadores *cross-encoder*.^[10]
- **RAGAS**: Framework de avaliação de sistemas RAG, que fornece métricas sem referência (*reference-free*) para avaliar fidelidade, relevância de respostas e qualidade da recuperação.^[18]
- **FAISS**: Biblioteca de busca eficiente em vetores densos da Meta AI, utilizada como backend de indexação em diversas implementações RAG de referência.

RAG e IA agêntica

Com a emergência dos agentes de IA autônomos, o RAG passou a desempenhar um papel central na dotação de agentes de memória de longo prazo e acesso a conhecimento especializado. Em arquiteturas agênticas, o RAG funciona como a "memória de longo prazo" de um agente, permitindo que ele consulte dinamicamente documentos, históricos de conversas e bases de conhecimento durante a execução de tarefas complexas e de múltiplos passos.^{[11][12]}

O Protocolo de Contexto de Modelo (MCP), padrão aberto introduzido pela Anthropic em 2024 e doado à Linux Foundation em 2025, define como agentes de IA se conectam a fontes de dados que podem alimentar sistemas RAG, incluindo bancos de dados, repositórios de documentos e APIs externas. O primitivo **Resources** do MCP é especialmente relevante nesse contexto, pois fornece acesso estruturado a informações que servem como contexto para o modelo, de forma análoga ao que um recuperador RAG faz dentro do pipeline.^[19]

Sistemas multiagente modernos combinam RAG com capacidades de planejamento e execução de ferramentas, permitindo que agentes não apenas recuperem informações relevantes, mas também planejem sequências de ações e executem tarefas no ambiente digital de forma autônoma.^[carece de fontes?]

Histórico e evolução

Trabalhos precursores (2019–2020)

Antes do artigo fundador de Lewis et al. (2020), pesquisadores já exploravam a combinação de recuperação e geração. O trabalho de Guu et al. (2020) sobre REALM (*Retrieval-Augmented Language Model Pre-Training*) propôs integrar a recuperação durante o pré-treinamento do modelo. Karpukhin et al. (2020) desenvolveram o DPR (*Dense Passage Retriever*), que se tornaria o recuperador padrão utilizado no artigo fundador do RAG.^[18]

Artigo fundador (2020)

O artigo "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks", de Lewis et al., foi publicado nos Anais da NeurIPS 2020. O trabalho demonstrou que um modelo seq2seq pré-treinado (baseado em BART) combinado com um índice vetorial denso de artigos da Wikipédia conseguia superar o estado da arte em múltiplos benchmarks de pergunta e resposta de domínio aberto, ao mesmo tempo em que permitia a atualização do índice de conhecimento sem retreinamento do modelo.^[1]

Popularização e adoção em larga escala (2022–2023)

A partir de 2022, com a popularização dos LLMs de uso geral como ChatGPT e Claude, o RAG tornou-se a técnica mais amplamente adotada para fundamentar respostas de IA em fontes verificáveis. Em 2023, houve um pico significativo de publicações acadêmicas e adoções industriais, com modelos RAG dominando vários benchmarks do KILT e tarefas de pergunta e resposta em poucos exemplos (*few-shot QA*). Frameworks como LangChain e LlamaIndex democratizaram a implementação de sistemas RAG para desenvolvedores sem experiência em aprendizado de máquina.^[18]

Maturidade e variantes avançadas (2024–2026)

Entre 2024 e 2026, o campo de RAG se expandiu significativamente com o desenvolvimento de variantes especializadas como GraphRAG (Microsoft, 2024), RAPTOR, RAFT e Self-RAG. Novas ferramentas de avaliação como RAGAS e o corpus RAGTruth permitiram análise mais granular de alucinações em outputs de sistemas RAG. O campo evoluiu de uma técnica única para uma família de métodos com centenas de publicações acadêmicas e dezenas de serviços comerciais.^[18]

Ver também

- Modelo de linguagem de grande escala
- Inteligência artificial generativa
- Agente inteligente
- Ciclo de Desenvolvimento de Software Aumentado por IA
- Model Context Protocol
- LangChain
- Engenharia de prompts

- Aprendizado de máquina
- Processamento de linguagem natural
- Alucinação (inteligência artificial)
- Banco de dados

Referências

1. Lewis, Patrick; et al. (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks» (<https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf>) (PDF). *Advances in Neural Information Processing Systems (NeurIPS)*. **33**
2. Klesel, M.; Wittmann, H. F. (2025). «Retrieval-Augmented Generation (RAG)». Springer. *Business & Information Systems Engineering*. **67** (4): 551–561. doi:10.1007/s12599-025-00945-3 (<https://dx.doi.org/10.1007/s12599-025-00945-3>)
3. «What Is Retrieval-Augmented Generation aka RAG» (<https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/>). 2023, atualizado em 2025. NVIDIA Blog. Consultado em 22 de março de 2026
4. «RAG is a way of improving LLM performance, blending the LLM process with a web search or document look-up» (<https://arstechnica.com>). Ars Technica. 2024. Consultado em 22 de março de 2026
5. «Can a technology called RAG keep AI models from making stuff up?» (<https://arstechnica.com/ai/2024/06/can-a-technology-called-rag-keep-ai-models-from-making-stuff-up/>). Ars Technica. 6 junho 2024. Consultado em 7 março 2025
6. «Retrieval Augmented Generation (RAG)» (<https://www.promptingguide.ai/techniques/rag>). Prompt Engineering Guide. Consultado em 22 de março de 2026
7. «Retrieval Augmented Generation (RAG) for LLMs» (<https://www.promptingguide.ai/research/rag>). Prompt Engineering Guide. Consultado em 22 de março de 2026
8. Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; Karpukhin, Vladimir; Goyal, Naman; Küttler, Heinrich; Lewis, Mike; Yih, Wen-tau; Rocktäschel, Tim; Riedel, Sebastian; Kiela, Douwe (6 dezembro 2020). *Retrieval-augmented generation for knowledge-intensive NLP tasks* (<https://dl.acm.org/doi/abs/10.5555/3495724.3496517>). International Conference on Neural Information Processing Systems. Red Hook, NY, USA: Curran Associates Inc. ISBN 978-1-7138-2954-6. Consultado em 9 dezembro 2025
9. «Information retrieval» (<https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-information-retrieval>). Microsoft. 10 janeiro 2025. Consultado em 15 março 2025
10. Diversos autores (2025). «Evaluating Retrieval-Augmented Generation Variants for Clinical Decision Support». *Electronics (MDPI)*. **14** (21). doi:10.3390/electronics14214227 (<https://dx.doi.org/10.3390/electronics14214227>)
11. «Why Google's AI Overviews gets things wrong» (<https://www.technologyreview.com/2024/05/31/1093019/why-are-googles-ai-overviews-results-so-bad/>). MIT Technology Review. 31 maio 2024. Consultado em 7 março 2025
12. Lewis, Patrick; Perez, Ethan; Piktus, Aleksandra; Petroni, Fabio; Karpukhin, Vladimir; Goyal, Naman; Küttler, Heinrich; Lewis, Mike; Yih, Wen-tau; Rocktäschel, Tim; Riedel, Sebastian; Kiela, Douwe (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks» (<https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>). Curran Associates, Inc. *Advances in Neural Information Processing Systems*. **33**: 9459–9474. arXiv:2005.11401 (<https://arxiv.org/abs/2005.11401>)
13. «GraphRAG: Unlocking LLM discovery on narrative private data» (<https://www.microsoft.com/en-us/research/blog/graphrag-unlocking-llm-discovery-on-narrative-private-data/>). Microsoft Research Blog. 2024. Consultado em 22 de março de 2026

14. Xu; et al. (2025). «MEGA-RAG: a retrieval-augmented generation framework with multi-evidence guided answer refinement for mitigating hallucinations of LLMs in public health». PMC. *Frontiers in Artificial Intelligence*. doi:10.3389/frai.2025 (https://dx.doi.org/10.3389%2Ffrai.2025)
15. Diversos autores (2025). «Retrieval augmented generation for large language models in healthcare: A systematic review». PMC. *PLoS ONE / PMC*
16. Lee, Kenton; Chang, Ming-Wei; Toutanova, Kristina (2019). « "Latent Retrieval for Weakly Supervised Open Domain Question Answering" » (https://aclanthology.org/P19-1612.pdf) (PDF)
17. Wang, Boxin; Ping, Wei; Xu, Peng; McAfee, Lawrence; Liu, Zihan; Shoeybi, Mohammad; Dong, Yi; Kuchaiev, Oleksii; Li, Bo; Xiao, Chaowei; Anandkumar, Anima; Catanzaro, Bryan (2023). «Shall We Pretrain Autoregressive Language Models with Retrieval? A Comprehensive Study» (https://aclanthology.org/2023.emnlp-main.482/). *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. [S.l.: s.n.] pp. 7763–7786. doi:10.18653/v1/2023.emnlp-main.482 (https://dx.doi.org/10.18653%2Fv1%2F2023.emnlp-main.482) 
18. Diversos autores (2026). «A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions». *Journal of Future Artificial Intelligence and Technologies*. doi:10.54517/faith.v1i2.297 (https://dx.doi.org/10.54517%2Ffaith.v1i2.297)
19. Khattab, Omar; Zaharia, Matei (2020). «ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT» (https://dl.acm.org/doi/10.1145/3397271.3401075). *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. [S.l.: s.n.] pp. 39–48. ISBN 978-1-4503-8016-4. doi:10.1145/3397271.3401075 (https://dx.doi.org/10.1145%2F3397271.3401075)

Leitura adicional

Artigo fundador

- Lewis, Patrick; et al. (2020). «Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks» (https://proceedings.neurips.cc/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf) (PDF). *Advances in Neural Information Processing Systems (NeurIPS)*. **33**

Livros

- Rothman, Denis (2024). *RAG-Driven Generative AI: Build Custom Retrieval Augmented Generation Pipelines with LlamaIndex, Deep Lake, and Pinecone*. [S.l.]: Packt Publishing
- Huyen, Chip (2025). *AI Engineering: Building Applications with Foundation Models*. [S.l.]: O'Reilly Media. ISBN 978-1-098-16610-1 Verifique | isbn= (ajuda)

Artigos acadêmicos

- Klesel, M.; Wittmann, H. F. (2025). «Retrieval-Augmented Generation (RAG)». *Springer. Business & Information Systems Engineering*. **67** (4): 551–561. doi:10.1007/s12599-025-00945-3 (https://dx.doi.org/10.1007%2Fs12599-025-00945-3)

- Diversos autores (2026). «A Systematic Review of Key Retrieval-Augmented Generation (RAG) Systems: Progress, Gaps, and Future Directions». *Journal of Future Artificial Intelligence and Technologies*. doi:10.54517/faith.v1i2.297 (<https://dx.doi.org/10.54517%2Faith.v1i2.297>)
- Guu, K.; et al. (2020). «REALM: Retrieval-Augmented Language Model Pre-Training» (<https://arxiv.org/abs/2002.08909>). *ICML 2020*
- Asai, A.; et al. (2023). «Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection» (<https://arxiv.org/abs/2310.11511>). *arXiv*

Recursos técnicos

- What Is Retrieval-Augmented Generation aka RAG — NVIDIA Blog (<https://blogs.nvidia.com/blog/what-is-retrieval-augmented-generation/>)
- Retrieval Augmented Generation (RAG) — Prompt Engineering Guide (<https://www.promptingguide.ai/techniques/rag>)
- Retrieval-Augmented Generation (RAG) — Springer Business & Information Systems Engineering (<https://link.springer.com/article/10.1007/s12599-025-00945-3>)
- Artigo original de Lewis et al. (2020) no arXiv (<https://arxiv.org/abs/2005.11401>)

Obtida de "[https://pt.wikipedia.org/w/index.php?title=Geração_aumentada_por_recuperação&oldid=72138346](https://pt.wikipedia.org/w/index.php?title=Gera%C3%A7%C3%A3o_aumentada_por_recupera%C3%A7%C3%A3o&oldid=72138346)"